

Sketch-to-CAD: Generating Computer Aided Design Models from Hand-Drawn Sketches

Ghadi Nehme

Department of Mechanical Engineering
Massachusetts Institute of Technology

ghadi@mit.edu

Abstract

Computer-Aided Design (CAD) is an essential tool in engineering, enabling the creation of precise 3D models. However, converting 2D hand-drawn sketches into CAD designs remains a challenging and time-consuming task. This work introduces a Sketch-to-CAD framework that uses a CNN encoder for extracting spatial features from sketches and a Transformer decoder for generating sequential CAD commands. Leveraging transfer learning, the encoder was pre-trained on sketch reconstruction tasks, resulting in enhanced feature extraction capabilities. The proposed framework was evaluated on the DeepCAD dataset, achieving high accuracy in predicting CAD commands and parameters from hand-drawn sketches. Latent space analysis revealed that the encoder effectively captured meaningful geometric representations, with logical clustering based on object dimensions and shapes. Saliency analyses further validated the model’s interpretability, confirming its reliance on structural sketch features for decision-making. Despite these achievements, challenges remain in accurately predicting parameters related to scale and translation, often leading to misaligned or scaled outputs.

1. Introduction

Computer-Aided Design (CAD) is a prominent technology utilized across various engineering disciplines to facilitate 3D design, development, and simulation analysis. A singular CAD file comprises an amalgamation of operations, such as 2D sketches, extrudes, fillets, and chamfers. This CAD file effectively establishes a chronological framework, enabling editors to make upstream adjustments that seamlessly propagate throughout the design. While 3D design plays a pivotal role in prototyping and product development, the expeditious nature of 2D sketching empowers engineers to swiftly brainstorm and explore numerous design concepts. Consequently, it is customary to transform a 2D sketch into a CAD design. However, the process of

converting a hand-drawn 2D sketch into a comprehensive 3D model is arduous and time-consuming. By developing a pipeline for 3D design that provides an adjustable model timeline, one can facilitate the refinement of a 3D model, thereby enhancing the accessibility of 3D design and presenting an opportunity to revolutionize global workflows. In this context, we propose an encoder-decoder framework to generate CAD models from hand-drawn sketches, using a CNN encoder for feature extraction and a Transformer decoder for sequential CAD commands. Leveraging transfer learning, we pretrain the encoder on sketch reconstruction tasks to enhance its feature extraction capabilities, enabling effective automation of CAD generation from sketches.

2. Related Work

Prior works have explored the use of sketching in the context of Computer Aided Design (CAD). One of the seminal works in this area is “Sketch2CAD: Sequential CAD Modeling by Sketching in Context” [1], which develops an autoencoder latent space for generating CAD timelines for a 3D design. Subsequently, a generative adversarial network (GAN) is developed using the latent-GAN technique to randomly produce vectors in the latent space that result in realistic 3D CAD timelines. Additionally, a dataset of 178,238 CAD models and timelines was generated with designs that exclusively use simple sketch and extrude commands.

Another related work is “A 2D Sketch-Based User Interface for 3D CAD” [2], which presents a user interface that allows users to search for and retrieve 3D CAD models based on sketches. This system typically uses similarity metrics to match input sketches with existing models in a database.

Despite modern efforts to develop models for CAD specific applications, previous generative models for 3D design focus on non-customizable formats [3]. OpenAI’s Point-E produces point-cloud 3D outputs from complex prompts [4]. While point clouds, meshes, and .stl files enable finer and printable representations of 3D volumes, software for adjusting such designs does not exist.

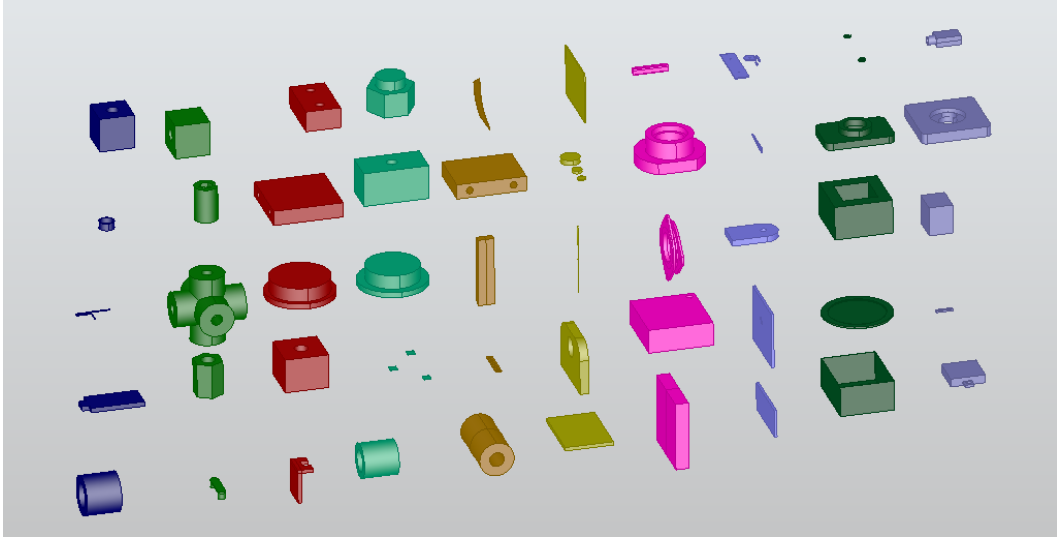


Figure 1. CAD models associated to the CAD sequences in the dataset.

3. Dataset and Features

We utilize a subset of the DeepCAD Dataset, which consists of 175,308 CAD models [5]. This dataset is derived from the ABC dataset [6], a collection of one million CAD models. Each CAD model in the DeepCAD Dataset is represented as a sequence of steps for sequentially building the sketches and extrusions of the model. (Fig. 1)

3.1. Input: Hand-Drawn Sketches

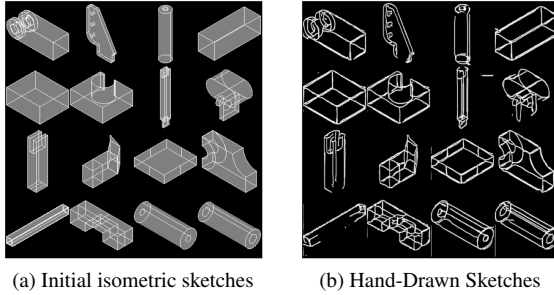


Figure 2. Example of isometric sketches obtained after using pythonocc-core and their corresponding hand-drawn sketches.

The input of our model is a hand-drawn sketch of the CAD file. To obtain it, we start by generating isometric sketches of the CAD models using the pythonocc-core library [7], loading each model’s vector representation. Each image is then inverted to make sketched lines the positive signal. We normalize the image to a range between 0 and 1. Starting with an initial size of 1024×1024 , we resize it to 128×128 to minimize input size (Fig. 2a). We then apply the method from Photo-Sketching: Inferring Contour Drawings from Images [8] to produce a hand-drawn style

sketch (Fig. 2b). Finally, we one-hot encode each pixel in the image, designating edges and background as two separate classes.

3.2. Output: Steps of CAD model

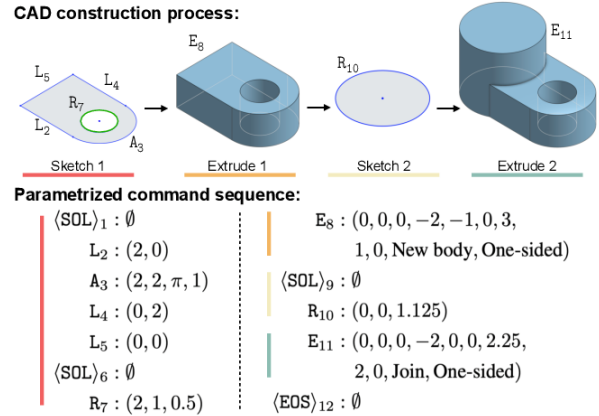


Figure 3. A CAD model example specified by the commands in Figure 5. (Top) the CAD model’s construction sequence, annotated with the command types. (Bottom) the command sequence description of the model. [5]

A CAD model, denoted as C , is defined by a series of curve commands and extrusion commands (Fig. 3). In other words, C can be represented as a command sequence $C = [C_1, \dots, C_{N_c}]$, where each C_i takes the form (t_i, p_i) . Here, t_i represents the command type, and p_i represents the corresponding parameters. The commands and parameters are detailed in Figure 5.

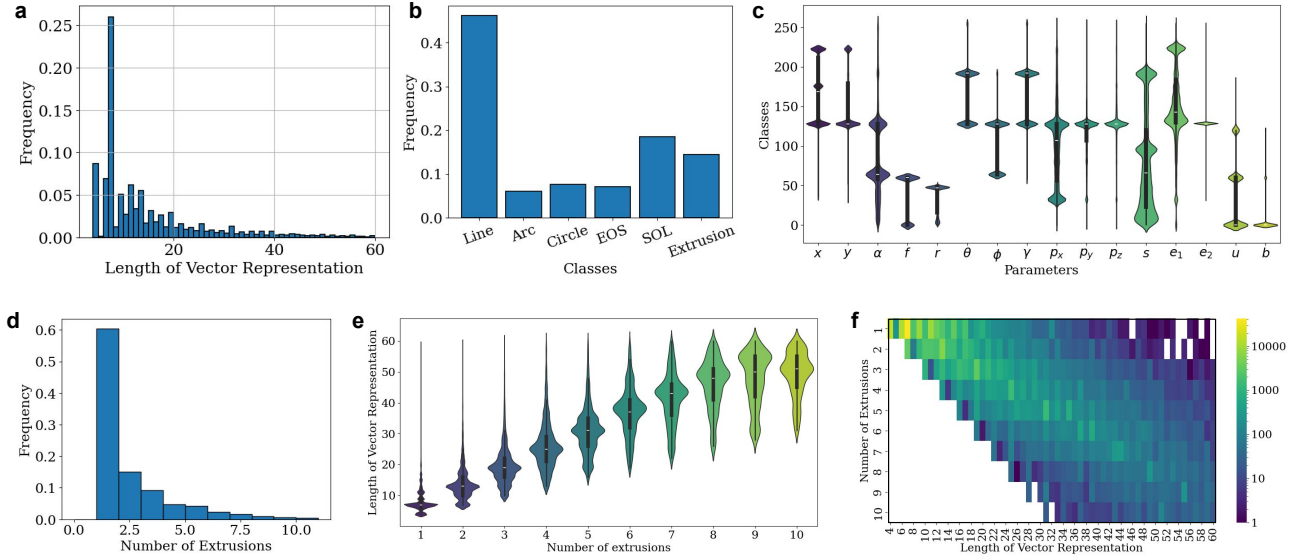


Figure 4. **Quantitative analysis of extrusion counts, parameters and command types class distribution, and vector representation lengths in our dataset.** (a) Histogram of vector representation lengths highlights clustering at shorter lengths. (b) Class occurrence among command types, with 'Line' as the most frequent element. (c) Violin plot of the variation of parameter distributions across classes, showing distinct clusters for each parameter. (d) Frequency distribution of extrusion numbers reveals a predominance of designs with minimal extrusions. (e) Violin plots illustrate the relationship between vector length and extrusion count, showing progressive lengthening with additional extrusions. (f) Heatmap depicting the correlation between extrusion count and vector length, with intensity indicating frequency.

Commands	Parameters
$\langle \text{SOL} \rangle$	$\emptyset$
L (Line)	x, y : line end-point
A (Arc)	x, y : arc end-point α : sweep angle f : counter-clockwise flag
R (Circle)	x, y : center r : radius
E (Extrude)	θ, ϕ, γ : sketch plane orientation p_x, p_y, p_z : sketch plane origin s : scale of associated sketch profile e_1, e_2 : extrude distances toward both sides b : boolean type, u : extrude type
$\langle \text{EOS} \rangle$	$\emptyset$

Figure 5. CAD commands and their parameters. $\langle \text{SOL} \rangle$ indicates the start of a loop; $\langle \text{EOS} \rangle$ indicates the end of the sequence. [5]

After normalization steps, the final network-friendly representation of the CAD step is a 17×1 vector: $[t_i, x, y, \alpha, f, r, \theta, \phi, \gamma, p_x, p_y, p_z, s, e_1, e_2, u, b]$. Here, t_i is an integer between 0 and 5, and the other elements are in-

tegers between -1 and 255. A value of -1 indicates that the specific element is not used in that step. Representing the continuous variables of a CAD instruction (ie: x, y , or z , location) allows expressing each values with 8-bit numbers.

Additionally, when creating CAD designs, it is important to maintain certain geometric relationships such as parallel and perpendicular sketch lines. However, directly generating continuous parameters through parameter regression can lead to errors that violate these relationships. To overcome this, parameter quantization is used to categorize parameters into specific levels, allowing the network to better uphold learned geometric relationships.

For this project, we will limit the number of steps to $N_c = 32$ and pad all step sequences to 32 using the empty step: $\langle \text{EOS} \rangle$.

Thus, we have one classification task for the CAD commands with the number of classes equals 6 and 16 other classification tasks for the CAD arguments with the number of classes equal 256. We can see in Figures 4.b and 4.c, the data distribution across the classes.

We observe a large data imbalance in the dataset. There are more Lines than any other CAD step (Fig. 4.b). We also observe that for almost each parameter, there are a few dominant classes that can be observed with the peaks in the violin plots in Figure 4.c.

Similarly, we note an imbalance in the number of extrusions and the length of vector representations, with shorter vector representations and fewer extrusions being more frequent (Fig. 4.a, d, e).

4. Methods

Our goal is to generate CAD models from hand-drawn sketches. To achieve this, we propose an encoder-decoder framework where the encoder leverages Convolutional Neural Networks (CNNs), and the decoder employs a Transformer-based architecture. CNNs are well-suited for the encoder as they excel at extracting hierarchical spatial features from images, making them ideal for capturing the structural and geometric patterns in sketches. This allows the encoder to generate a compact and informative latent representation of the input sketch. On the other hand, Transformers are effective for the decoder due to their ability to model sequential data and long-range dependencies, making them ideal for predicting the sequential commands and parameters required to generate CAD models.

We will enhance the encoder’s feature extraction capabilities through transfer learning by pre-training an autoencoder for sketch-based feature representation. Various encoder architectures and initialization strategies will be explored, including a frozen ResNet18, a pretrained ResNet18, and our custom CNN encoder pre-trained on sketch reconstruction tasks.

Additionally, we will develop a separate encoder-decoder model to generate CAD models from isometric CAD drawings. Finally, we will compare the performance of these two approaches using hand-drawn sketches and isometric drawings as inputs.

4.1. Model Architectures

4.1.1 Convolutional Neural Networks Encoder

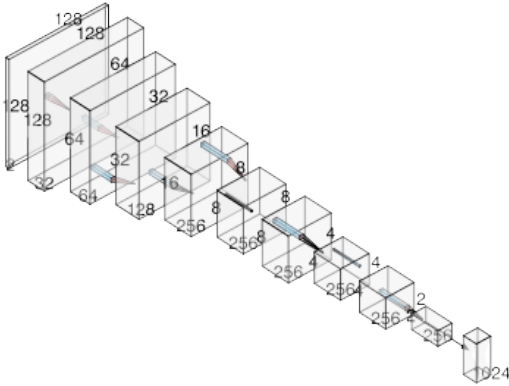


Figure 6. CNN Encoder Model Architecture.

CNNs are widely used for image-related tasks due to their ability to capture spatial dependencies in the data. We designed a CNN-based architecture that takes the input sketch image and learns to extract relevant features for predicting the CAD steps. The CNN architecture consists of multiple convolutional layers followed by a fully connected layer (Fig. 6).

4.1.2 Transformers Decoder

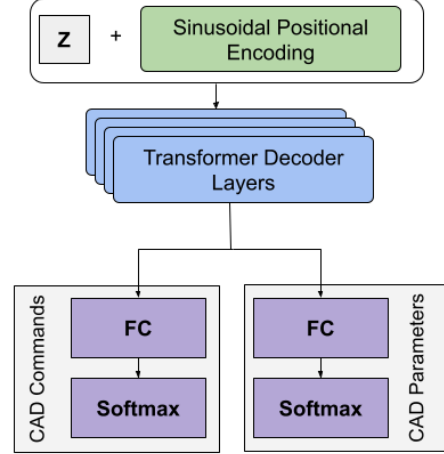


Figure 7. Transformer Decoder Model Architecture.

Built on Transformer blocks, the decoder (Figure 7) processes the latent vector z generated by the encoder and predicts the sequence of CAD commands. We add to z sinusoidal positional encoding before passing it to the decoder. Output from the final Transformer block is passed through fully connected layers to predict the CAD command sequence $\hat{C} = [\hat{C}_1, \dots, \hat{C}_{N_c}]$, including the command type $\hat{t}_i$ and parameters $\hat{p}_i$.

4.1.3 Autoencoders for Transfer Learning

To initialize the weights of the CNN encoder, we pre-trained two autoencoders: one for hand-drawn sketches and one for isometric sketches. These autoencoders consist of symmetric encoder-decoder architectures trained to reconstruct input sketches, ensuring the encoder learns high-quality feature representations. These pre-trained models serve as the initial weights of the CNN encoder for the main task, facilitating effective learning during the joint training of the CNN encoder and Transformer decoder. (Figure 8)

4.2. Evaluation

To evaluate our models, we calculate the Cross Entropy Loss between the ground truth sequence of CAD steps and

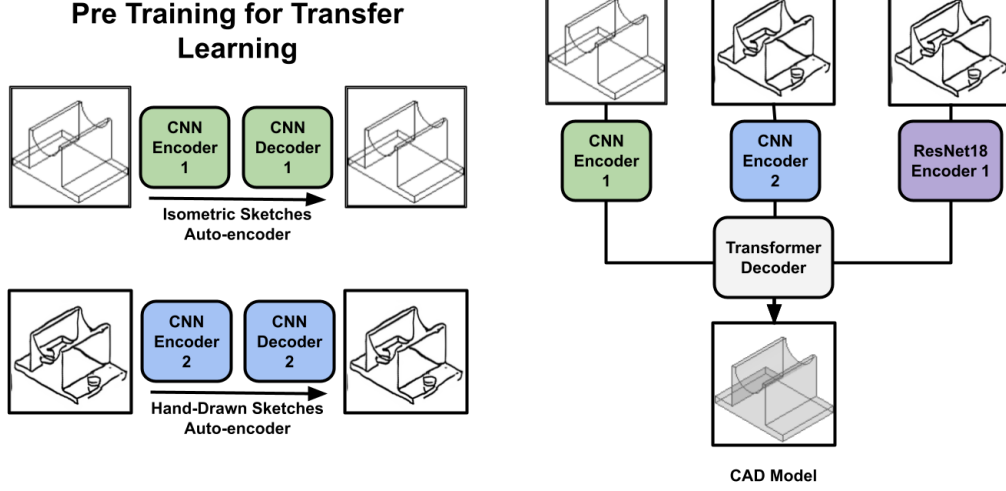


Figure 8. Sketch-to-CAD pipeline using transfer learning with pre-trained CNNs and a Transformer decoder to predict CAD commands from sketches.

the predicted sequence:

$$\mathcal{L} = \sum_{i=1}^{N_c} \ell(\hat{t}_i, t_i) + \beta \sum_{i=1}^{N_c} \sum_{j=1}^{N_P} \ell(\hat{\mathbf{p}}_{ij}, \mathbf{p}_{ij}),$$

where $\ell(\cdot, \cdot)$ is the standard Cross-Entropy loss, N_p is the number of parameters (16), and β is a hyperparameter balancing the importance of the two terms. Contributions from unused commands and parameters (labeled -1) are excluded using a masking mechanism.

Model evaluation includes qualitative and quantitative components:

- **Accuracy:** We compute the accuracy of the predicted command types and parameters compared to the ground truth:

$$\text{Accuracy}_t = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{1}(\hat{t}_i = t_i),$$

$$\text{Accuracy}_p = \frac{1}{N_c N_P} \sum_{i=1}^{N_c} \sum_{j=1}^{N_P} \mathbf{1}(\hat{t}_i = t_i) \mathbf{1}(\hat{\mathbf{p}}_{ij} = \mathbf{p}_{ij}).$$

- **Percentage of Invalid CAD Sequences:** The percentage of CAD models that cannot be converted to STEP files is computed.
- **Cross Entropy Loss:** Loss values for commands and parameters are calculated using the formula above.
- **Saliency Analysis:** Saliency plots are generated for a subset of the test set to visualize input regions that most influence predictions.

- **Latent Space Analysis:** We use PCA to project latent vectors z into a lower-dimensional space (2D), enabling visualization of clustering patterns and class separability.
- **Qualitative Evaluation:** The generated CAD sequences are visualized to assess realism, focusing on timelines and extrusions.

4.3. Model Implementation and Training

We initiated our experiments by training CNN autoencoders for both hand-drawn sketches and isometric drawings. These autoencoders were trained for 50 epochs using the Binary Cross-Entropy loss function, with a learning rate of 0.001 and a learning rate scheduler incorporating both warmup and plateau adjustments. The pretrained encoders derived from these autoencoders served as initialization for subsequent models.

Four different models were developed and trained:

- CNN Encoder with Transformer Decoder using Isometric Drawings as input
- CNN Encoder with Transformer Decoder using Hand-drawn Sketches as input
- Frozen ResNet18 Encoder with Transformer Decoder using Hand-drawn Sketches as input (Sketch-to-CAD)
- Pretrained ResNet18 Encoder with Transformer Decoder using Hand-drawn Sketches as input

The CNN encoders in the first two models were initialized with the pretrained weights from their corresponding autoencoders. All models were trained on a shuffled dataset, with 90% allocated for training and the remaining 10% evenly split for validation and testing.

We utilized the Adam optimizer and applied early stopping based on validation loss to mitigate overfitting. Training was conducted for up to 200 epochs with a batch size of 256. The learning rate was set at 0.0001 and featured a warmup phase during the initial training period.

The hyperparameters used for training the CNN-Transformer models are detailed in Table 1.

Table 1. Hyperparameters of the CNN | Transformer model.

Hyperparameter	Value
Latent space dimension d_{model}	1024
Initial Learning rate α	0.0001
Transformer: Number of heads h	16
Transformer: Number of layers h	4
Relative importance of loss terms β	2

5. Results and Discussion

5.1. Sketches Autoencoders

We first train two autoencoders to reconstruct the hand-drawn sketches and isometric drawings. The encoder consists of CNN layers and the decoder of upsampling CNN layers. After training the models, we get a training loss of 0.105, validation loss of 0.106 and a test loss of 0.108 for the isometric drawings autoencoder and a training loss of 0.112, validation loss of 0.117 and a test loss of 0.116 for the hand-drawn sketches autoencoder. We reconstruct some samples from the test set and show the results in Figure 9. We observe that we get a good reconstruction of the images and a low BCE loss, implying that the encoder was able to extract relevant features from the sketches.

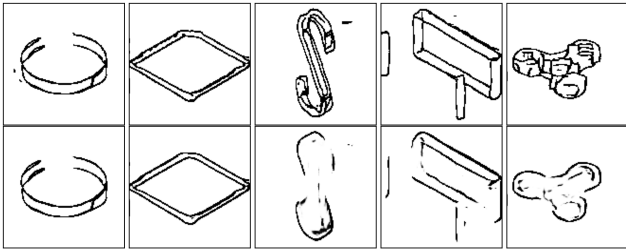


Figure 9. Sample of sketches from the test set (top) and their reconstructions (bottom) using the hand-drawn sketches autoencoder.

5.2. Performance of Different Encoders

The training, validation, and test accuracies for the command and parameters are noted in Table 2 for the different models we trained.

We observe that the *Frozen ResNet18* [9] achieves the lowest performance across all metrics, particularly with a higher invalid ratio (6.12%). This result is expected, as

the frozen encoder extracts general-purpose features that may not be optimal for the specific task of CAD command and parameter prediction from sketches. Unfreezing the ResNet18 encoder and initializing it with pretrained weights improved the model’s ability to extract relevant features, leading to a notable increase in performance metrics. However, the best results were achieved with our custom CNN architecture, which was initialized with a pretrained encoder trained on sketch reconstruction. This architecture effectively captures latent features, demonstrating the benefits of task-specific feature learning and transfer learning for this application.

5.3. Comparison Between Hand-Drawn Sketches and Isometric Drawings

Comparing the performance of the best model on hand-drawn sketches to the model trained on isometric drawings reveals a clear decay in performance. The custom CNN trained on isometric sketches achieved a test accuracy of 94.68% for t_i and 88.68% for p_i , while the same model trained on hand-drawn sketches achieved slightly lower test accuracies of 91.31% and 85.80% for t_i and p_i , respectively (Table 2). This decrease is expected, as hand-drawn sketches lack the precision and detail provided by isometric CAD sketches.

5.4. Saliency Plots

Saliency plots of the hand-drawn sketches provide a concise and informative visualization of the pixel components that have the greatest influence on the softmax logits. These plots offer a robust and clear representation of the relative importance of different pixels in determining the final output.

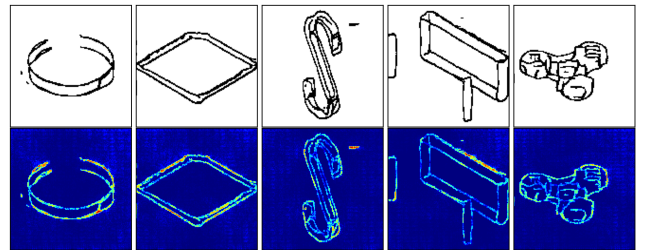


Figure 10. Saliency visualizations of the Sketch-to-CAD model highlighting the ability of the model to extract the features of the input sketch.

The saliency visualizations in Figure 10 clearly show that the model focuses on the edges and contours of the sketches to make predictions. This behavior aligns with the model’s task, as the structural boundaries are critical for understanding the geometry and features of the objects. The consistent emphasis on edges across different sketches

Table 2. CAD Commands t_i and Parameters p_i Accuracies and the invalid ratio for the different models we trained.

Model Encoder	Input Sketch	Train $t_i \uparrow$	Train $p_i \uparrow$	Val $t_i \uparrow$	Val $p_i \uparrow$	Test $t_i \uparrow$	Test $p_i \uparrow$	Invalid Ratio (%) $\downarrow$
CNN	Isometric	95.78	91.20	93.52	89.78	94.68	88.68	2.24
Frozen ResNet18	Hand-Drawn	86.07	78.15	85.65	72.47	84.03	72.55	6.12
Pretrained ResNet18	Hand-Drawn	90.39	83.54	89.26	78.63	88.16	76.09	4.45
CNN (Sketch-to-CAD)	Hand-Drawn	92.26	86.85	89.53	84.82	91.31	85.80	4.03

indicates that the model effectively captures the essential features needed for CAD model prediction. These visualizations validate the interpretability of the model, confirming it relies on logical and meaningful aspects of the input sketches.

5.5. Latent Space Analysis

To analyze the latent space representation, we encode 512 input images from the test set using the trained CNN encoder of Sketch-to-CAD to obtain the latent vectors. We then apply Principal Component Analysis (PCA) to reduce the dimensionality of these vectors to two principal components, which are plotted in Figure 11, with each image placed at its corresponding location in the PCA space.

The results demonstrate that the model has learned a

meaningful representation of the data, as the images are grouped in a logical and interpretable manner. A clear trend is observed along the second principal component: as its value increases, the length of the CAD models along the z-direction also increases. Similarly, the first principal component reveals a relationship with the object dimensions in the x- and y-directions. As the value of the first component increases, the length of the model along the x-direction (pointing towards $[-1, 1]$ in the images) decreases, while the length along the y-direction (pointing towards $[1, 1]$) increases. This results in thinner objects being positioned at the top-left and top-right of the PCA plot, with more uniformly sized objects clustering in the center.

Additionally, distinct clusters emerge in the PCA space: box-like models tend to group in the bottom-left, while

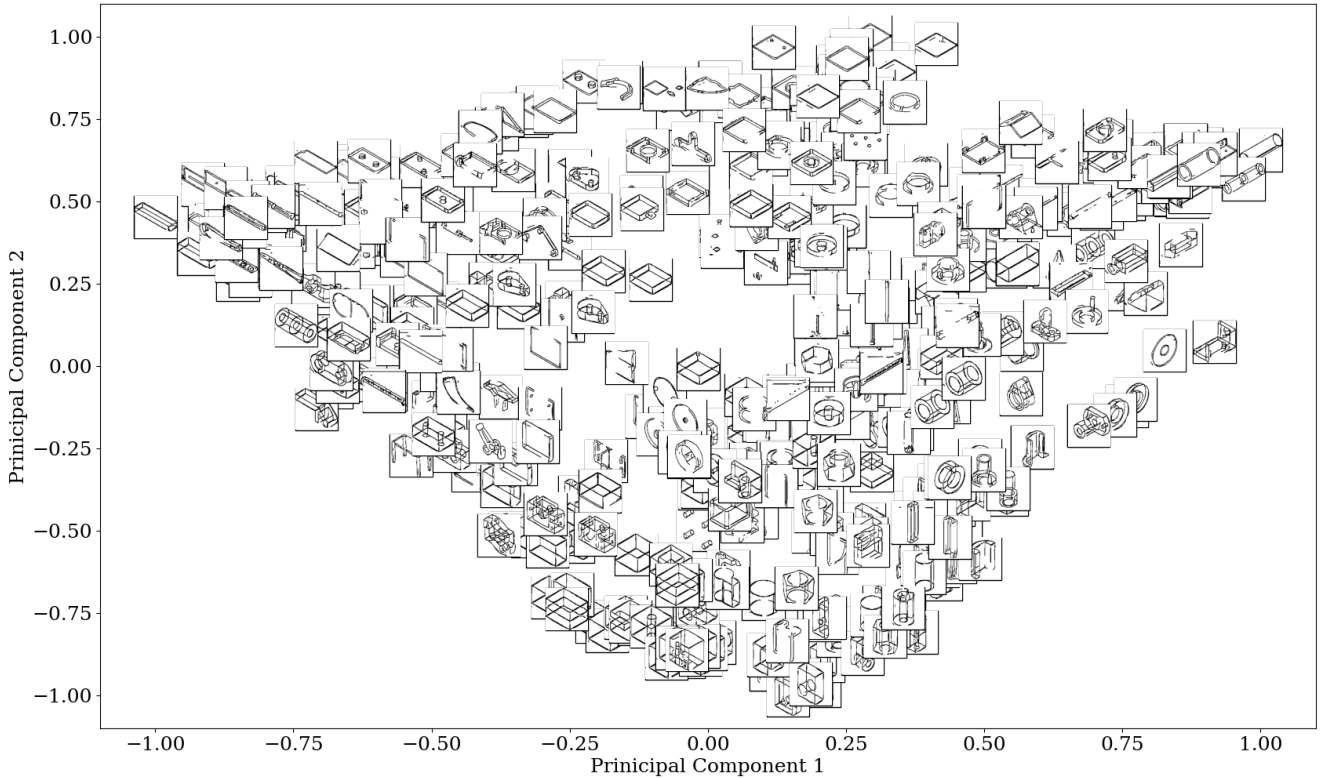


Figure 11. Visualization of the Latent Space Representation Using PCA: Input Sketches Placed at Their Corresponding Principal Component Coordinates.

cylindrical or circular models are found predominantly in the bottom-right. This clustering and spatial organization within the latent space provide strong evidence that the encoder has effectively captured meaningful geometric and structural features of the input sketches, enabling the model to generate CAD representations with high accuracy.

5.6. CAD Generation

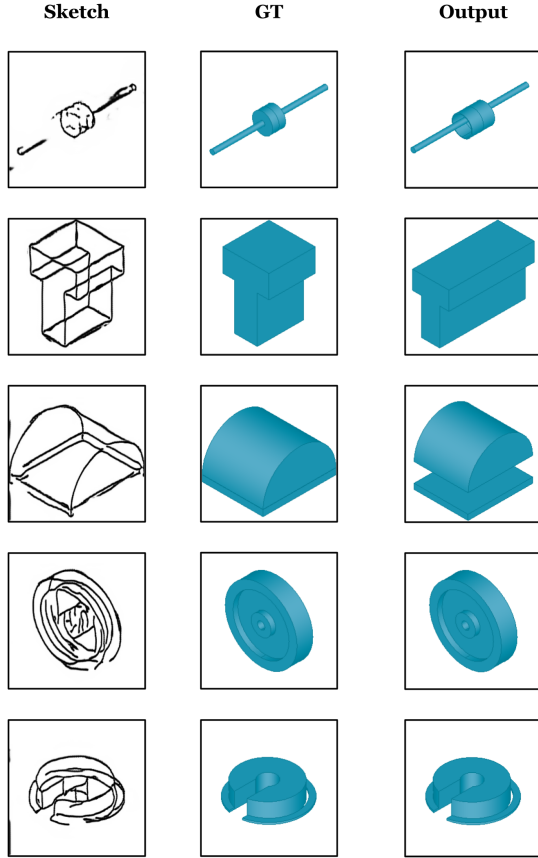


Figure 12. Successful and unsuccessful CAD models generated using Sketch-to-CAD, with the input sketch and the ground truth CAD. The first three are unsuccessful CAD models and the last two are successful CAD models.

After finetuning the Sketch-to-CAD model, we visualize examples of CAD generated at test time using hand-drawn sketches. Successful and unsuccessful CAD outputs depicted in Figure 12 were results of the CNN | Transformer model, initialized with the pretrained encoder on sketch reconstruction. Successful output means that the output sequence is the same as the ground truth.

By visually inspecting the generated CAD steps, we observe that the model was able to generate complex CAD models from sketches. Nonetheless, we observe some limitations.

First, similarly to the first CAD part in Figure 12, the model generates a CAD model that looks like the input sketch, but doesn't overlap with the ground truth. In fact, in the sketch, we see all the lines of the part even the ones that are supposed to be hidden. Thus, the model taking these lines into account generates a part which has slightly different features than the ground truth.

Moreover, if we look at the second CAD part in Figure 12, we observe that the model was able to generate a part very similar to the ground truth but scaled up and translated. This unsuccessful part is mainly due to the inability of the model of finding the right scale and coordinate system based only on a sketch. You could predict the relative scaling of different features of the sketch, but it is impossible to guess the scale of the entire part only from sketch. This leads us to the conclusion that we need to introduce new metrics which are translation and scale invariant with respect to the ground truth and the output.

Finally, similarly to the third CAD part in Figure 12, we observe that the model predicted the right components of the CAD part but they were not well positioned with respect to each other. In fact, the model doesn't have access to the coordinate of the plane where the first feature was extruded and thus, isn't able to choose the appropriate relative position of the next feature. A solution to this issue would be to pass from one step to another the origin coordinates to have a reference on where to place the different components of the CAD part.

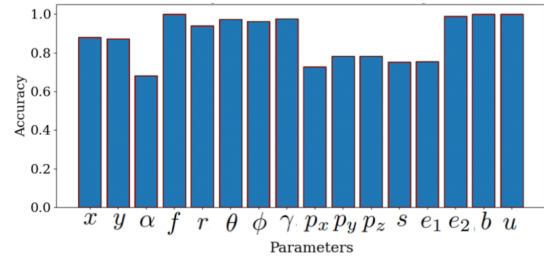


Figure 13. Parameters Accuracies for the Sketch-to-CAD Model on the test set.

Now, we decide to take a closer look at the accuracies of each of the 16 parameters predicted for the best model we got using the hand-drawn sketches, the CNN encoder - Transformer Decoder model. We observe that many parameters have very high accuracies, while others barely reach 60-70%. We see that parameters related to coordinates like the point coordinates (x , y) and the plane origin coordinates (p_x , p_y , p_z), and the parameter related to scaling s are the ones which are associated with the lowest accuracies (Fig. 13). This confirms that the failure we observe in our CAD parts earlier is indeed due to this inability of extracting coordinates of key points or the scaling of the different components by simply looking at a 2D sketch.

5.7. Limitations

While the CNN-Transformer model showed promising results, there are several limitations to consider:

CAD step representation: The CAD steps were represented as a sequence of discrete symbols in this study. This representation may introduce challenges in capturing fine-grained details and continuous variations in the CAD steps.

Good CAD models associated with low metrics: Sketch-to-CAD was able to generate complex CAD parts from sketches. Some outputs looked the same as the ground truth but translated or scaled up/down which associated them to low accuracy while they should be associated to a high accuracy. This highlights the need to **develop new evaluation metrics** for this specific application.

5.8. Future Directions

To address the limitations and further improve the Sketch-to-CAD model, several future directions can be explored:

Larger and more diverse dataset: Collecting a larger and more diverse dataset of CAD models and corresponding sketches would enhance the models' ability to generalize to a wider range of design scenarios and improve their performance.

Fine-grained CAD representations: Investigating alternative representations for CAD steps, such as continuous or vector-based representations, could capture finer details and provide more flexibility in generating accurate CAD outputs.

Deep Reinforcement Learning: It would be interesting to train a Deep RL model which, based on a current state: CAD steps, is able to take an action: a CAD step, and get a reward based on the new state. Some potential reward functions could be one that penalizes failing CAD sequences and one that encourages CAD sequences that generate sketches similar to the input image. In fact, this approach makes more sense in this context, given that there exist infinitely many representations that work for one sketch and thus, having a labeled CAD sequence for an image limits the capacity of the model to generalize.

6. Conclusion

In this report, we introduced a Sketch-to-CAD framework that generates CAD models from hand-drawn sketches. By utilizing a CNN encoder to extract spatial features and a Transformer decoder to predict sequential CAD commands, our model was able to generate CAD models from hand-drawn sketches. Through transfer learning, the CNN encoder was pre-trained on sketch reconstruction tasks, enhancing its ability to capture meaningful latent representations. Quantitative evaluations demonstrated the framework's strong performance in generating accurate

CAD models, while saliency analyses confirmed its reliance on structural sketch features for decision-making.

Our latent space analysis revealed that the model learns logical and interpretable geometric features, with distinct clustering patterns corresponding to variations in object dimensions and shapes. However, the model exhibited challenges in predicting parameters related to scaling and coordinates, often leading to CAD outputs that were scaled, translated, or misaligned compared to the ground truth. These limitations highlight the need for more robust metrics that account for translation and scale invariance to better evaluate CAD generation quality.

Looking forward, future work can focus on addressing the identified limitations by expanding datasets, exploring fine-grained CAD step representations, and integrating reinforcement learning techniques for more flexible and generalized CAD generation. These efforts, coupled with improved evaluation metrics, can further enhance the applicability of Sketch-to-CAD models in real-world design scenarios, ultimately contributing to the advancement of sketch-based CAD modeling workflows.

References

- [1] Changjian Li, Hao Pan, Adrien Bousseau, and Niloy J Mitra. Sketch2cad: Sequential cad modeling by sketching in context. *ACM Transactions on Graphics (TOG)*, 39(6):1–14, 2020. 1
- [2] Jiantao Pu, Kuiyang Lou, and Karthik Ramani. A 2d sketch-based user interface for 3d cad model retrieval. *Computer-aided design and applications*, 2(6):717–725, 2005. 1
- [3] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion, 2022. 1
- [4] Alex Nichol, Heewoo Jun, Pratul Dhariwal, Pamela Mishkin, and Mark Chen. Point-e: A system for generating 3d point clouds from complex prompts, 2022. 1
- [5] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6772–6782, 2021. 2, 3
- [6] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019. 2
- [7] Thomas Paviot. pythonocc, 2022. 2
- [8] Mengtian Li, Zhe Lin, Radomir Mech, Ersin Yumer, and Deva Ramanan. Photo-sketching: Inferring contour drawings from images. In *2019 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 1403–1412. IEEE, 2019. 2
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 6